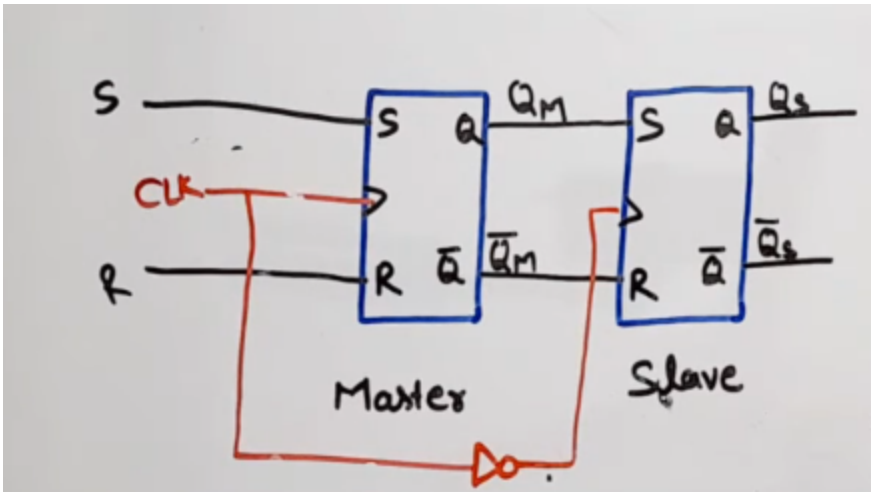
The Master-Slave SR Flip-Flop

n## 1. The Transparency Problem and the Master-Slave Solution

As discussed previously, a standard Clocked SR Latch is **level-sensitive** (transparent). As long as the clock remains High (`1`), any changes to the S and R inputs will immediately pass through and change the output. In complex, looped circuits, this transparency causes unstable feedback oscillations.

To solve this, we must completely isolate the inputs from the final outputs. We do this by chaining two Clocked SR Latches together in a **Master-Slave** configuration.



The Architecture

- **The Master:** The first gated latch. It receives the external S and R inputs and the raw Clock signal.
- **The Slave:** The second gated latch. It receives the outputs of the Master (Q_m and $\overline{Q}_m$) as its inputs. Crucially, **the clock signal is inverted** before it reaches the Slave.

2. Principle of Operation

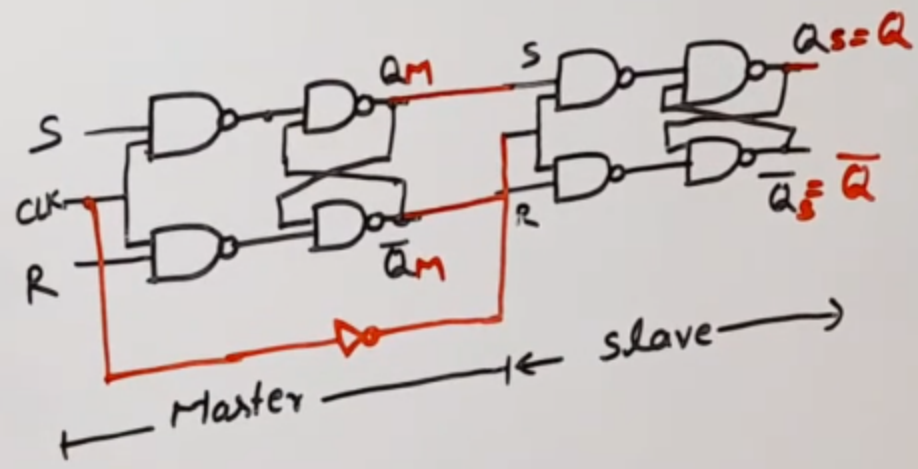
Because the clock is inverted between the two stages, only one latch can be "open" (transparent) at any given time.

1. **When Clock is High ($Clk = 1$):** The Master latch is enabled. It accepts the S and R inputs and updates its internal state (Q_m).
   - The inverted clock going to the Slave is `0`. The Slave is disabled, locking the final output (Q_s) safely in its previous state. The circuit is actively loading data, but the output is shielded.
2. **When Clock is Low ($Clk = 0$):**

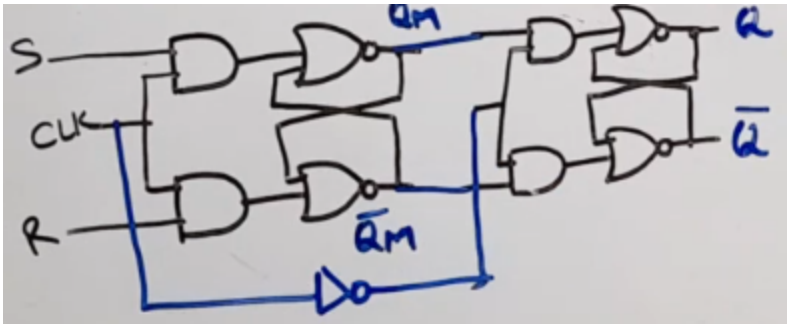
- The Master latch is disabled. It stops accepting external inputs and locks its current Q_m value.
- The inverted clock going to the Slave becomes `1`. The Slave wakes up, reads the locked Q_m value from the Master, and updates the final output (Q_s).

This mechanism creates **Edge-Triggered** behavior. From the outside, it appears that the final output (Q_s) only ever changes at the exact moment the clock falls from `1` to `0` (the negative edge).

Circuit Diagram (NAND Implementation)



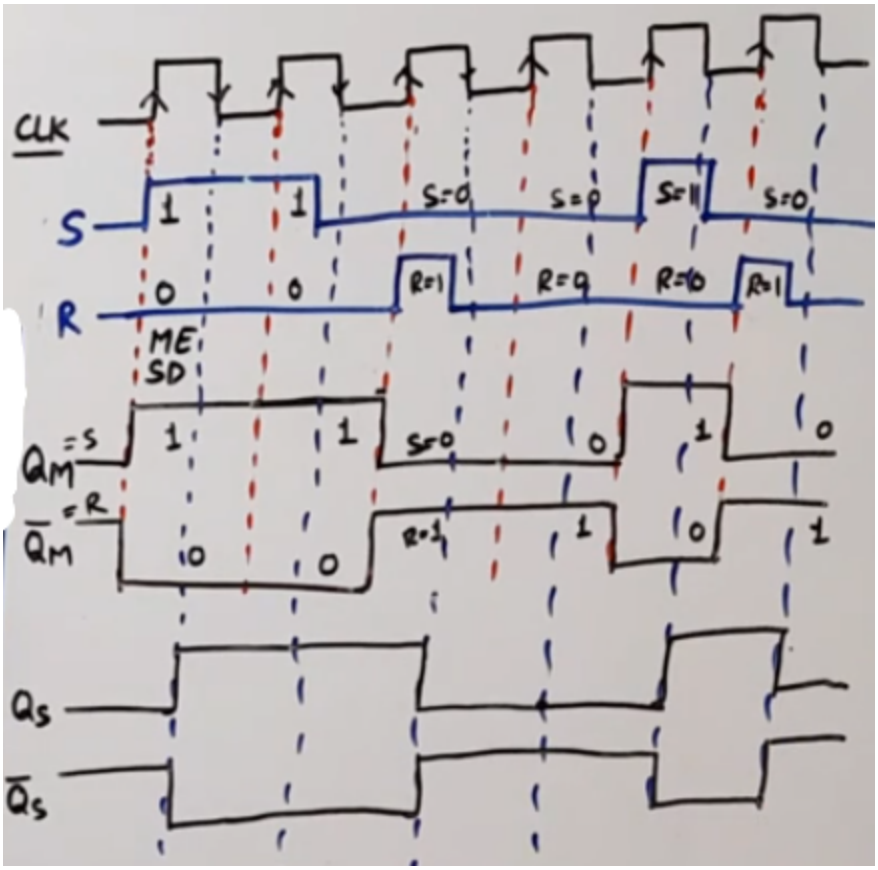
Circuit Diagram (NOR Implementation)



3. Timing Diagram Analysis

To truly understand a Master-Slave flip-flop, you must analyze its waveforms over time.

- **S and R:** The user inputs.
- **Q_m (Master Output):** Tracks the inputs *only* while the clock is High.
- **Q_s (Slave / Final Output):** Only updates to match Q_m exactly at the falling edge of the clock (when it transitions High $\rightarrow$ Low).



This ensures that any chaotic signals or rapid switching on the S and R lines during a clock cycle are safely absorbed by the Master and never accidentally leak to the rest of the computer system.